Final Project Sharing

Customized ISA Processor APR

iclab042 楊子睬
iclab043 陳燁
iclab046 劭品祥

Observation

Performance:

- **Total Cycle x Clock Period x Area: 25%**
- **Total Cycle x Clock Period x Average Power x Peak Power: 25%**

- Total cycle x clock period still matter
- Too aggressive reducing latency will increase cycle time and power
- Too aggressive reducing power will increase cycle time and latency (傷害area的公式)
- 所以我選了一個兩者都可以拿不錯的分數的design

	Area Perf.	Rank1	Power Perf.	Rank2	Total R	
iclab042	1	169.8861961	9	10		total rank 1
iclab043	2	171.4200882	10	12		total rank 2

Overview

- Multi-cycle CPU
- Merge I-Cache and D-Cache (384X16 SRAM)
- Write-back cache with periodic flush (start -end dirty pointer)
- Single execution and memory unit shared across both cores
- Shift-add multiplier
- Clock gating
- SRAM CS control

- My performance
  - Post sim Cycle time: 4.1 ns
  - Chip Area: 471957.3
  - Latency: 45441
  - Total power:0.009146
  - Peak power:0.0997

Observation on pattern (instruction access)

- Almost mimimum DRAM access
- less access count on some cache blocks(lower the cache block size)

```
========================================
instr 0
MISS COUNT: 41
inst 0: 00, 80, 100, 180, 200, 280,
300, 380, 4ea, 5ea, 600, 5a0, 628, 680,
874, 9fe, a00, b1c, c38, d5e, e68, f70,
f80, 10a0, 11ae, 1200, 1280, 13b8, 14be,
15d6, 16e8, 17f0, 1800, 1900, 1a1c, 1b34,
1c38, 1d3c, 1e40, 1f60, 1ff0
```

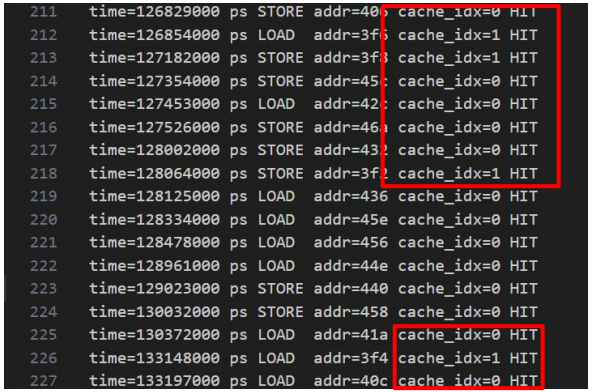
```
========================================
instr 1
MISS COUNT: 39
inst 1: 00, 80, 1cc, 2cc, 3cc, 4e8, 5f0,
6f8, 700, 806, 90c, a10, b1a, c26, d26,
e26, f28, 102a, 1080, 1100, 1228, 1328,
142e, 1536, 1636, 1742, 1780, 1800, 193e,
1a3e, 1b40, 1c02, 1c80, 1d00, 1d80, 1e00,
1e80, 1f00, 1f80
```

```
========================================
instr 0
CACHE BLOCK USAGE (64 entries/block)
block 0000: 62
block 0080: 64
block 0100: 64
block 0180: 64
block 0200: 64
block 0280: 64
block 0300: 64
block 0380: 54
block 0480: 1
block 0580: 17
block 0600: 57
block 0680: 22
block 0800: 3
block 0980: 1
block 0a00: 15
block 0b00: 15
block 0c00: 20
block 0d00: 6
block 0e00: 5
block 0f00: 8
block 0f80: 17
block 1080: 8
block 1180: 41
block 1200: 64
block 1280: 29
block 1380: 4
block 1480: 13
block 1580: 10
block 1680: 5
block 1780: 8
block 1800: 1
block 1900: 15
block 1a00: 13
block 1b00: 3
block 1c00: 3
block 1d00: 3
block 1e00: 17
block 1f00: 15
block 1f80: 8
```

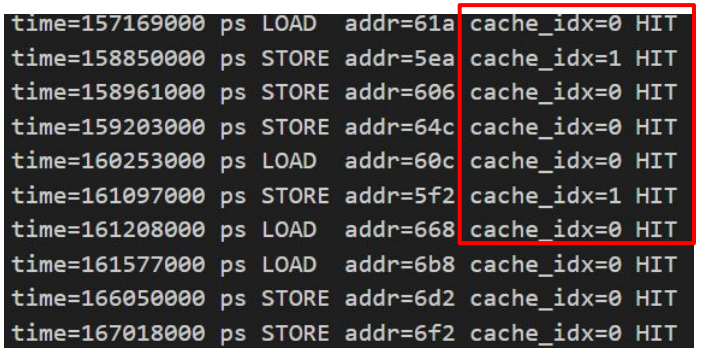
```
========================================
instr 1
CACHE BLOCK USAGE (64 entries/block)
block 0000: 64
block 0080: 39
block 0180: 1
block 0280: 1
block 0380: 15
block 0480: 5
block 0580: 5
block 0680: 4
block 0700: 4
block 0800: 4
block 0900: 3
block 0a00: 6
block 0b00: 7
block 0c00: 1
block 0d00: 1
block 0e00: 2
block 0f00: 2
block 1000: 43
block 1080: 64
block 1100: 21
block 1200: 1
block 1300: 4
block 1400: 5
block 1500: 1
block 1600: 7
block 1700: 31
block 1780: 64
block 1800: 32
block 1900: 1
block 1a00: 2
block 1b00: 1
block 1c00: 63
block 1c80: 64
block 1d00: 64
block 1d80: 64
block 1e00: 64
block 1e80: 64
block 1f00: 64
block 1f80: 64
```

Observation on pattern (data access)

- Minimum DRAM access for two 128-entry direct map data cache.
- No redundant fetch.
- Test pattern frequently alternates access between block index 0 and 1, so the cache cannot be reduced to a single-entry block.




```
========================================
data cache MISS COUNT: 10
data: 42, 130, 206, 330, 406, 518, 61a, 7ac, 86c, 908
```



I-Cache + D-Cache

- Cache hierarchy
  - Address 0–63 : Core 0 instruction cache
  - Address 64–127 : Core 1 instruction cache
  - Address 128–383 : Shared data cache
- 1-entry instruction cache
- 2-entry direct-map write-back data cache
- Write-back condition: instruction counter reaches 49 or PC hits 0x1FFE
- Write-back range: dirty start ~ end pointer

Combinations	Area
two 128X16 SRAM	~11000
two 256X16 SRAM	~13600
one 512X16 SRAM	96105.96
one 384X16 SRAM	83288.34
one 256X16 SRAM	68199.15

Clock gating

- Register Bank Clock Gating
  - The 16 registers in two cores are gated individually.
  - Clock is enabled only during the WRITE stage.
  - Only the target register of the active core receives the gated clock.
  - Enable condition: WRITE stage + active core + target register selected

```
// ====================================================
// core_1 registers
// ====================================================
wire core_1_r0_clk_en = core_1_write_en && (wb1_dec[0] || wb2_dec[0]);
wire core_1_r0_gclk = clk & core_1_r0_clk_en;
always @(posedge core_1_r0_gclk or negedge rst_n) begin
    if(!rst_n) begin
        core_1_r0 <= 16'd0;
    end
    else begin
        if(wb1_dec[0])
            core_1_r0 <= write_target1_data_wb;
        else if(wb2_dec[0])
            core_1_r0 <= write_target2_data_wb;
    end
end
```

Clock gating

- Multiplier Clock Gating
  - Internal registers such as busy, shift_cnt, acc, shifted_A, booth_B, result, and done use gated clocks
  - When the multiplier is idle, these registers stop toggling.
- Booth Algorithm
  - Used Radix-4 Booth encoding to process 2 bits per cycle.
  - Each cycle selects one operation based on booth_B[2:0]: +0, +A, +2A, -A, or -2A.

```
assign busy_clk_en      = start_mult || busy || done_r;
assign cnt_clk_en       = start_mult || busy || done_r;
assign acc_clk_en       = start_mult || busy || done_r;
assign shifted_A_clk_en = start_mult || busy || done_r;
assign booth_B_clk_en   = start_mult || busy || done_r;
assign result_clk_en    = start_mult || busy || done_r;
assign done_clk_en      = start_mult || busy || done_r;

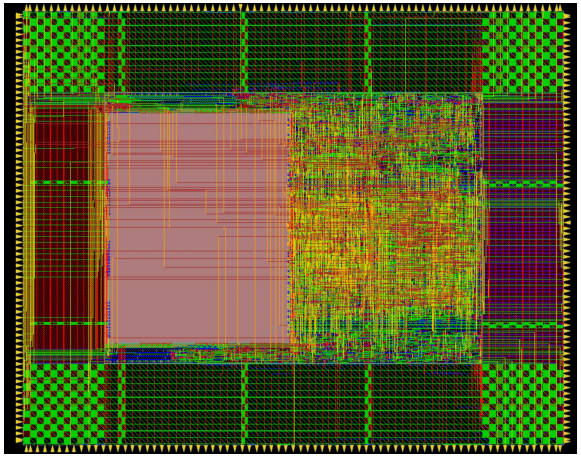
assign busy_gclk      = clk & busy_clk_en;
assign cnt_gclk       = clk & cnt_clk_en;
assign acc_gclk       = clk & acc_clk_en;
assign shifted_A_gclk = clk & shifted_A_clk_en;
assign booth_B_gclk   = clk & booth_B_clk_en;
assign result_gclk    = clk & result_clk_en;
assign done_gclk      = clk & done_clk_en;
```

SRAM CS control

- SRAM CS Control
  - sram_CS is asserted only when SRAM actually needs to be accessed:
  - sram_CS = sram_read_access | sram_write_access
  - Read access is enabled during:
    - Dirty block writeback read
    - Cache hit read
    - READ_CACHE output cycle
  - Write access is enabled during:
    - AXI refill writing data into SRAM
    - D-cache write hit / write-after-refill

APR Optimization

- Reduce IO boundary
  - Lower Peak power -> Lower core ring width (I set 9.24, 6 pairs)
  - Lower core ring offset (I set 1)
  - -> boundary 115.6
- Reduce area: rise core utilization
  - higher strip distance -> lower strip count (only 5 strips)
  - lower SRAM halo (5.0)
  - -> core utilization 0.9 set in floor plain without error

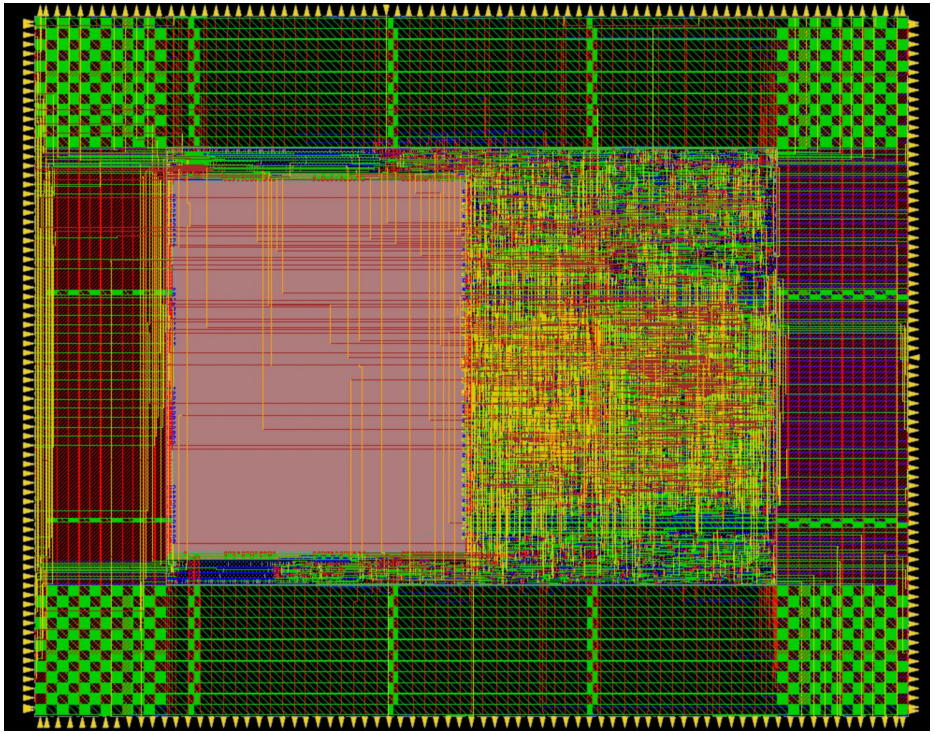


```
floorPlan -site core_5040 -r 0.73 0.9 115.6 115.6 115.6 115.6
```

Conclusion

- Multi-cycle CPU
- OPtimized cache size (384X16 SRAM)
- Write-back cache with periodic flush (start -end dirty pointer)
- Clock gating
- SRAM CS control
- APR area optimization

- My performance
  - Post sim Cycle time: 4.1 ns
  - Chip Area: 471957.3
  - Latency: 45441
  - Total power:0.009146
  - Peak power:0.0997



Thank You